HANDBOOK

2026 EDITION

THE AI EVALUATION METRICS HANDBOOK

77 metrics. What each one means, how to calculate it, and a worked example for every single one.

RAG | AGENTS | REGRESSION | RL | VISION | SQL | + MORE

Compiled by **@ai.with.etqad**

CONTENTS

Every AI system needs metrics that match what it is supposed to do. This handbook explains each metric from the reel: what it means, how to calculate it, a small worked example, and when to reach for it.

00	**Start here: the four outcomes**	basics	3
01	**Classification**	6 metrics	4
02	**Anomaly Detection**	3 metrics	6
03	**Regression and Forecasting**	8 metrics	8
04	**Retrieval, Search and Recommendation**	8 metrics	11
05	**RAG Generation**	4 metrics	14
06	**Summarization and Translation**	8 metrics	16
07	**Chatbots and Agents**	10 metrics	19
08	**Code Generation and Text-to-SQL**	8 metrics	22
09	**Reinforcement Learning**	5 metrics	25
10	**Computer Vision: Object Detection and Image Generation**	8 metrics	27
11	**Clustering**	4 metrics	30
12	**Production AI**	5 metrics	32

f(x) Formula, typeset **EXAMPLE** Worked example with real numbers ! Trap or limitation to watch

00 Start here: the four outcomes

Most classification, detection and retrieval metrics are built from four counts. Picture a spam filter judging 100 emails.

Outcome	Meaning	Spam filter example
True Positive (TP)	Predicted positive, actually positive	Spam correctly sent to spam
False Positive (FP)	Predicted positive, actually negative	A real email wrongly sent to spam
True Negative (TN)	Predicted negative, actually negative	A real email correctly kept in inbox
False Negative (FN)	Predicted negative, actually positive	Spam that slipped into the inbox

The running example used below: 100 emails, 20 of them spam. The filter flags 18 emails; 15 are truly spam. So TP = 15, FP = 3, FN = 5, TN = 77.

Two questions sit behind almost everything that follows. When the model says yes, how often is it right? That is precision. Of all the real yeses, how many did it catch? That is recall.

01 Classification

IN THIS SECTION Accuracy | Precision | Recall | F1 Score | ROC-AUC | Confusion Matrix

> For a classifier, report precision, recall and F1 together, and never accuracy alone on imbalanced data.

Accuracy

The share of all predictions that were correct. Simple, but misleading when one class dominates.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

EXAMPLE (15 + 77) / 100 = 0.92. The trap: a lazy model that calls every email "not spam" still scores 0.80 while catching zero spam.

Precision

When the model says positive, how often is it right? High precision means few false alarms.

$$\text{Precision} = \frac{TP}{TP + FP}$$

EXAMPLE 15 / (15 + 3) = 0.83. Prioritise it when a false positive is expensive, such as blocking a legitimate payment.

Recall (Sensitivity, True Positive Rate)

Of all actual positives, how many did the model catch? High recall means few misses.

$$\text{Recall} = \frac{TP}{TP + FN}$$

EXAMPLE 15 / (15 + 5) = 0.75. Prioritise it when a miss is expensive, such as failing to flag cancer on a scan.

F1 Score

The harmonic mean of precision and recall. It is only high when both are high, so a model cannot game it by maximising one.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

EXAMPLE 2 × (0.83 × 0.75) / (0.83 + 0.75) = 0.79. The general F-beta score lets you weight recall β times as much as precision; F2 is common in medical and fraud work.

For more than two classes, compute per class and then average. Macro-average treats every class equally; weighted-average weights each class by how often it appears; micro-average pools all TP, FP and FN first.

ROC-AUC

Most classifiers output a probability, and you pick a threshold to turn it into yes or no. The ROC curve plots True Positive Rate against False Positive Rate at every threshold. AUC is the area under that curve.

$$\text{TPR} = \frac{TP}{TP + FN} \qquad \text{FPR} = \frac{FP}{FP + TN}$$

Intuition: AUC is the probability that a randomly chosen positive gets a higher score than a randomly chosen negative. 0.5 is a coin flip, 1.0 is perfect separation. How to compute it: sweep the threshold from 1 to 0, record (FPR, TPR) at each step, and sum the area with the trapezoid rule, or call `sklearn.metrics.roc_auc_score`.

Confusion Matrix

Not a single number but the full grid of TP, FP, FN and TN (or an N × N grid for N classes). Rows are actual classes, columns are predicted classes. It shows exactly which classes the model confuses with each other, which no single score can.

	Predicted spam	Predicted not spam
Actual spam	15 (TP)	5 (FN)
Actual not spam	3 (FP)	77 (TN)

02 Anomaly Detection

IN THIS SECTION PR-AUC | False Positive Rate | Detection Latency

> **Anomalies are rare, often under 1% of data, so accuracy is useless here and precision-recall metrics take over. Precision, recall and F1 work exactly as in Classification, with the anomaly as the positive class.**

PR-AUC (Average Precision)

The area under the precision-recall curve, built by sweeping the anomaly-score threshold and plotting precision against recall at each step.

$$\mathrm{AP} = \sum_{n} (R_n - R_{n-1})\, P_n$$

Here P_n and R_n are precision and recall at the n-th threshold. Why it beats ROC-AUC here: with 10 fraud cases in 100,000 transactions, a model can post ROC-AUC of 0.95 while most of its alerts are false. PR-AUC exposes that, because precision collapses. A random model's PR-AUC equals the positive rate (0.0001 here), not 0.5.

False Positive Rate

The share of normal events wrongly flagged as anomalies.

$$\mathrm{FPR} = \frac{FP}{FP + TN}$$

EXAMPLE 99,990 normal transactions and 500 false alerts gives FPR = 0.5%. That sounds tiny, but it is 500 alerts for an analyst team to triage. Track it alongside alerts per day, which is what the on-call team actually feels.

Detection Latency

Time between when an anomaly starts and when the system flags it. Calculate it per incident, then report the mean or median.

$$\mathrm{Latency} = t_{\mathrm{detected}} - t_{\mathrm{onset}}$$

EXAMPLE A server starts leaking memory at 10:00 and the alert fires at 10:07, so latency is 7 minutes. A detector that catches everything an hour late can be worse than one that catches 80% within a minute.

03 Regression and Forecasting

IN THIS SECTION MAE | RMSE | R^2 | MAPE | sMAPE | MASE | Residual Analysis | Prediction Interval Coverage

> **When the output is a number, measure how far predictions land from reality. Forecasting uses the same core metrics plus a few that handle scale and uncertainty.**

Running example: actual house prices (in lakhs) are 100, 200 and 300; the model predicts 110, 190 and 330. The errors are +10, −10 and +30.

MAE (Mean Absolute Error)

The average size of the error, ignoring direction. Easy to explain, because it is in the same units as the target.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

EXAMPLE (10 + 10 + 30) / 3 = 16.7 lakhs. "On average we are off by about 17 lakhs."

RMSE (Root Mean Squared Error)

Square the errors, average them, take the square root. Squaring punishes big misses much harder than small ones.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

EXAMPLE √((100 + 100 + 900) / 3) = 19.1 lakhs. RMSE is above MAE because the 30-lakh miss dominates. A large gap between RMSE and MAE is a signal of occasional huge errors.

R^2 (Coefficient of Determination)

The share of variation in the target that the model explains, compared with always predicting the mean.

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

EXAMPLE Mean = 200, so the baseline squared error is 10,000 + 0 + 10,000 = 20,000. The model's is 1,100. R^2 = 1 − 1,100 / 20,000 = 0.945. 1.0 is perfect, 0 is no better than the mean, and it can go negative for a model worse than the mean.

MAPE (Mean Absolute Percentage Error)

The average error as a percentage of the actual value. Business teams love it because it is unit-free.

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^{n} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

EXAMPLE (10% + 5% + 10%) / 3 = 8.3%. Watch out: it explodes when actual values are near zero, and it penalises over-forecasts more than under-forecasts.

sMAPE (Symmetric MAPE)

A fix for MAPE that divides by the average of actual and forecast, so over- and under-forecasts are treated more evenly.

$$\text{sMAPE} = \frac{100\%}{n} \sum_{i=1}^{n} \frac{2\,|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}$$

EXAMPLE (20/210 + 20/390 + 60/630) / 3 = 8.1%. Bounded between 0% and 200%.

MASE (Mean Absolute Scaled Error)

Your forecast's MAE divided by the MAE of a naive forecast that just predicts the previous value. It answers: am I beating the dumbest possible baseline?

$$\text{MASE} = \frac{\text{MAE}_{\text{model}}}{\frac{1}{n-1}\sum_{t=2}^{n} |y_t - y_{t-1}|}$$

EXAMPLE The naive "same as last step" error on 100 → 200 → 300 is 100 per step, so MASE = 16.7 / 100 = 0.17. Below 1 beats the naive forecast; above 1 means the naive one wins. The denominator is normally computed on the training data. It works across series with different scales, which is why forecasting competitions use it.

Residual Analysis

Not one number but a check on the errors themselves. Plot residuals (actual – predicted) against predicted values and against time. Good residuals look like random noise around zero. A funnel shape means error grows with size; a curve means the model missed a non-linear pattern; a trend over time means drift.

Prediction Interval Coverage

Forecasts often come with a range, like "between 180 and 220 with 95% confidence". Coverage checks whether the actual value lands inside the range as often as promised.

$$\text{Coverage} = \frac{\#\{\, i : L_i \le y_i \le U_i \,\}}{n}$$

EXAMPLE 95% intervals that contain the actual value 47 times out of 50 give 94% coverage, which is well calibrated. 70% coverage means the model is overconfident. Report the average interval width too, because absurdly wide intervals hit 100% coverage while being useless.

04 Retrieval, Search and Recommendation

IN THIS SECTION | Precision@K | Recall@K | MRR | MAP | NDCG@K | Hit Rate@K | Coverage | Diversity

> **Ranking metrics ask two things: did the right items show up, and did they show up near the top? These are also the metrics for the retrieval half of a RAG system.**

Running example: a query has 4 relevant documents in the whole corpus. The system returns 5 results, and positions 1 and 3 are relevant: [✓, ×, ✓, ×, ×].

Precision@K

Of the top K results, what fraction are relevant?

$$\text{Precision@}K = \frac{\text{relevant items in top } K}{K}$$

EXAMPLE 2 / 5 = 0.40. Matters when users (or an LLM's context window) only see a few results and junk is costly.

Recall@K

Of all relevant items that exist, what fraction made it into the top K?

$$\text{Recall@}K = \frac{\text{relevant items in top } K}{\text{total relevant items}}$$

EXAMPLE 2 / 4 = 0.50. For RAG this is often the most important retrieval number: if the right chunk never reaches the LLM, no prompt can save the answer. It needs a labelled set of relevant documents per query.

MRR (Mean Reciprocal Rank)

For each query, find the rank of the first relevant result and take 1 / rank. Average across queries.

$$\text{MRR} = \frac{1}{|Q|}\sum_{q=1}^{|Q|}\frac{1}{\text{rank}_q}$$

EXAMPLE Three queries whose first relevant hit is at rank 1, rank 3, and nowhere: (1 + 1/3 + 0) / 3 = 0.44. Best when one good answer is enough, like search-and-click or question answering.

MAP (Mean Average Precision)

Average Precision (AP) rewards a query for placing every relevant item high. At each rank where a relevant item appears, take Precision@that rank, then average. MAP is the mean of AP across queries.

$$\text{AP} = \frac{1}{\text{total relevant}} \sum_{k=1}^{K} \text{Precision@}k \times \text{rel}(k)$$

EXAMPLE Relevant at ranks 1 and 3, so Precision@1 = 1.0 and Precision@3 = 0.67. AP = (1.0 + 0.67) / 4 = 0.42. (Some libraries divide by relevant items retrieved instead of total relevant, which would give 0.83; check which one you are using.)

NDCG@K (Normalized Discounted Cumulative Gain)

The go-to metric when relevance is graded (say 0 to 3) rather than yes or no. Each result earns its relevance score, discounted by a log of its position, so a great result at rank 1 counts far more than at rank 5. Divide by the score of the perfect ordering to land between 0 and 1.

$$\text{DCG@}K = \sum_{i=1}^{K} \frac{\text{rel}_i}{\log_2(i+1)} \qquad \text{NDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K}$$

EXAMPLE Graded relevance of the 5 results is [3, 0, 2, 0, 1]. DCG = 3/1 + 0 + 2/2 + 0 + 1/2.585 = 4.39. The ideal order [3, 2, 1, 0, 0] gives IDCG = 3 + 2/1.585 + 1/2 = 4.76. NDCG@5 = 4.39 / 4.76 = 0.92. A common variant uses 2^rel − 1 in the numerator to reward highly relevant items even more.

Hit Rate@K

The share of queries or users for whom at least one relevant item appears in the top K. Coarse but intuitive.

$$\text{HitRate@}K = \frac{\#\{\, q : \text{at least one relevant in top } K \,\}}{|Q|}$$

EXAMPLE Across 1,000 users, 620 had at least one item they later bought in their top 10 recommendations, so HitRate@10 = 0.62.

Coverage (Catalog Coverage)

What fraction of your catalog ever gets recommended to anyone?

$$\text{Coverage} = \frac{|\text{unique items recommended}|}{|\text{catalog}|}$$

EXAMPLE 2,000 of 50,000 products ever appear in recommendations, so coverage is 4%. Low coverage means a popularity bubble where the long tail never gets shown.

Diversity (Intra-List Diversity)

How different the items within one recommendation list are from each other. Compute the average pairwise distance (often 1 − cosine similarity of item embeddings) across all pairs in the list.

$$\text{ILD} = \frac{2}{K(K-1)} \sum_{i<j} (1 - \text{sim}(i, j))$$

EXAMPLE A list of 10 near-identical black t-shirts scores close to 0; a list mixing a t-shirt, jeans and sneakers scores higher. Accuracy metrics alone will happily push the t-shirt list.

05 RAG Generation

IN THIS SECTION | Faithfulness | Answer Relevance | Context Precision | Context Recall

> **A RAG system can fail in two places: retrieval brings back the wrong chunks, or the LLM misuses good chunks. Score both halves separately. Use the retrieval metrics above for the first half; the four below cover the second. They are usually computed by an LLM judge, as in frameworks such as RAGAS, so they work without hand-labelled relevance data. Exact implementations vary by framework.**

Running example. Question: "When was the Eiffel Tower completed and how tall is it?" Retrieved context says it was completed in 1889 and stands 330 m tall. The answer says: "It was completed in 1889, is 330 m tall, and was designed by Gustave Eiffel himself."

Faithfulness (Groundedness)

Does every claim in the answer come from the retrieved context? This is the main anti-hallucination metric. An LLM breaks the answer into atomic claims, then checks each one against the context.

$$\text{Faithfulness} = \frac{\text{claims supported by the context}}{\text{total claims in the answer}}$$

EXAMPLE The answer makes 3 claims. 1889 and 330 m are in the context; "designed by Gustave Eiffel himself" is not (and is in fact inaccurate, since engineers in his firm led the design). Faithfulness = 2 / 3 = 0.67. Note that faithfulness checks support, not truth: a claim copied from a wrong document still counts as faithful.

Answer Relevance

Does the answer actually address the question, without dodging or padding? One common method: have an LLM generate several questions that the answer would be a good reply to, embed them, and measure how similar they are to the real question.

$$\text{AnswerRelevance} = \frac{1}{N}\sum_{i=1}^{N} \cos\left(E_q, E_{g_i}\right)$$

Here E_q is the embedding of the original question and E_g_i is the embedding of the i-th generated question. Example: an answer that covers both completion date and height reverse-engineers into questions very close to the original, scoring around 0.9. An answer that only gives the height scores lower, because the regenerated questions ignore "when".

Context Precision

Are the relevant chunks ranked near the top of what was retrieved? It is a ranking-aware precision: an LLM judges each retrieved chunk as useful or not for answering, then Precision@k is averaged over the positions holding useful chunks.

$$\text{ContextPrecision@}K = \frac{\sum_{k=1}^{K} \text{Precision@}k \times v_k}{\text{useful chunks in top } K}$$

Here v_k is 1 if the chunk at rank k is useful, else 0. Example: 3 chunks retrieved, useful at ranks 1 and 3. Precision@1 = 1.0, Precision@3 = 0.67, so the score is (1.0 + 0.67) / 2 = 0.83. Moving the useful chunk from rank 3 to rank 2 would raise it to 1.0.

Context Recall

Did retrieval bring back everything needed for a complete answer? It requires a reference answer: split the reference into claims and check how many can be found in the retrieved context.

$$\text{ContextRecall} = \frac{\text{reference claims found in the context}}{\text{total reference claims}}$$

EXAMPLE The reference answer has 2 claims (1889, 330 m) and both appear in the context, so context recall is 1.0. If the height chunk had not been retrieved, it would drop to 0.5, and no amount of prompt tuning could fix the answer. Context precision and recall are the LLM-judged cousins of Precision@K and Recall@K.

06 Summarization and Translation

IN THIS SECTION ROUGE | BLEU | chrF | BERTScore | COMET | Factual Consistency | Coverage and Conciseness | LLM-as-a-Judge and Human Review

> **Text-generation metrics fall into three generations: word overlap (ROUGE, BLEU, chrF), embedding similarity (BERTScore), and learned or LLM judges (COMET, factual consistency, LLM-as-judge). Overlap metrics are cheap baselines; the newer ones track human judgement much better.**

ROUGE (summarization)

Recall-Oriented Understudy for Gisting Evaluation. Counts how many words or phrases from a human reference summary appear in the model's summary.

$$\text{ROUGE-N} = \frac{\sum \text{matching n-grams}}{\sum \text{n-grams in the reference}}$$

Variants: ROUGE-1 (single words), ROUGE-2 (word pairs), ROUGE-L (longest common subsequence, which respects word order without needing words to be adjacent). Libraries usually report precision, recall and F1 for each.

EXAMPLE Reference "the cat sat on the mat", candidate "the cat is on the mat". Five of the six reference words match, so ROUGE-1 recall is 5/6 = 0.83. The longest common subsequence is "the cat on the mat" (5 words), so ROUGE-L recall is also 0.83. Weakness: "the feline rested on the rug" means the same thing and scores near zero.

BLEU (translation)

Bilingual Evaluation Understudy. The precision mirror of ROUGE: what fraction of the candidate's n-grams (1 to 4 words) appear in the reference, combined as a geometric mean, with a penalty for translations that are too short.

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^{4} \tfrac{1}{4} \log p_n\right) \qquad BP = \begin{cases} 1 & c > r \\ e^{1-r/c} & c \le r \end{cases}$$

Here p_n is the clipped n-gram precision, c the candidate length and r the reference length. Clipping stops "the the the the" from scoring well. Scores run 0 to 100 in most tools; above 30 is usually understandable, above 50 is strong. Use sacreBLEU so scores are comparable across papers.

chrF (translation)

Character n-gram F-score. Instead of whole words, it compares character sequences (typically up to 6 characters), and weights recall twice as much as precision.

$$\mathrm{chrF}_\beta = (1+\beta^2)\frac{\mathrm{chrP}\cdot\mathrm{chrR}}{\beta^2\,\mathrm{chrP}+\mathrm{chrR}}, \quad \beta = 2$$

Why it matters: in Hindi, Tamil or German, "ladka" and "ladke" are different words to BLEU but share most characters. chrF gives partial credit, so it handles rich morphology far better.

BERTScore

Embeds every token of the candidate and reference with a model like BERT, then greedily matches each token to its most similar token on the other side using cosine similarity.

$$R_{\mathrm{BERT}} = \frac{1}{|x|}\sum_{x_i \in x} \max_{\hat{x}_j \in \hat{x}} \cos(x_i, \hat{x}_j) \qquad F_{\mathrm{BERT}} = \frac{2PR}{P+R}$$

Here x is the reference and x̂ the candidate; precision swaps their roles. "The feline rested on the rug" now scores high against "the cat sat on the mat", because the embeddings know the words are close in meaning.

COMET (translation)

A neural model trained to predict human quality ratings. It reads the source sentence, the translation and the reference together and outputs a score. There is no hand formula: the metric is the trained model, run through the `unbabel-comet` library. It is currently among the metrics best correlated with professional translators' judgements, and reference-free versions (COMET-QE / CometKiwi) can score a translation with no reference at all.

Factual Consistency

Does the summary say only things the source supports? Two common methods:

1. NLI-based: a natural-language-inference model checks whether each summary sentence is entailed by the source (e.g. SummaC).
2. QA-based: generate questions from the summary, answer them from both summary and source, and compare (e.g. QAGS).

$$\text{Consistency} = \frac{\text{summary sentences entailed by the source}}{\text{total summary sentences}}$$

EXAMPLE A 5-sentence summary of an earnings report where one sentence states revenue rose 12% instead of the reported 8% scores 4 / 5 = 0.80. A summary can have high ROUGE and still fail this.

Coverage and Conciseness (summarization)

Coverage: list the key points of the source (by hand or with an LLM), then count how many the summary includes. Covering 7 of 10 key points gives 0.70.

Conciseness: how much the summary compresses without padding. A simple version is the compression ratio (summary words / source words); a better one is an LLM judge rating redundancy and filler on a 1 to 5 scale. Coverage and conciseness pull against each other, so report both.

LLM-as-a-Judge and Human Review

A strong LLM scores outputs against a written rubric (e.g. 1 to 5 for accuracy, helpfulness, tone), or picks the better of two outputs (pairwise). Report the average score or the win rate.

! Calibrate it before trusting it: score 50 to 100 examples by hand, and check the judge agrees with humans (Cohen's kappa above 0.6 is a reasonable bar). Known biases: favouring longer answers, the first option shown, and its own model family. Swap answer order and average to reduce position bias. Human review stays the gold standard for nuance, tone and cultural fit, especially in translation.

07 Chatbots and Agents

IN THIS SECTION Response Relevance | Coherence | Conversation Completion Rate | Hallucination Rate | CSAT | Task Completion Rate | Tool-Call Accuracy | Trajectory Efficiency and Step Count | Cost per Task | Error Recovery Rate

> **Chatbots are judged turn by turn and conversation by conversation. Agents are judged on outcomes and on the path they took: an agent that succeeds after 40 tool calls and ₹200 of tokens may be worse than one that succeeds 5% less often in 6 calls.**

Response Relevance

Does each reply address what the user just asked, given the conversation so far? Usually scored by an LLM judge on a 1 to 5 rubric per turn, then averaged. Example: the user asks to change a delivery address and the bot explains the returns policy; that turn scores 1.

Coherence

Does the reply make logical sense and stay consistent with earlier turns? A judge checks for contradictions ("your order shipped" then "your order is pending"), forgotten context, and disordered reasoning. Report the average rubric score, plus the rate of conversations containing at least one contradiction.

Conversation Completion Rate

The share of conversations where the user's goal was met without handing off to a human or abandoning.

$$\text{Completion} = \frac{\text{conversations resolved by the bot}}{\text{total conversations}}$$

EXAMPLE 820 of 1,000 support chats resolved without escalation gives 82%. Define "resolved" carefully: a user who gives up and leaves is not a success, even though no escalation happened.

Hallucination Rate

The share of responses containing at least one claim that is false or unsupported by the system's knowledge sources.

$$\text{Hallucination Rate} = \frac{\text{responses with } \geq 1 \text{ unsupported claim}}{\text{total responses}}$$

EXAMPLE A judge finds invented policies or prices in 12 of 400 sampled replies, so the rate is 3%. For RAG-backed bots this is roughly 1 minus the share of fully faithful answers.

CSAT (Customer Satisfaction Score)

The classic post-chat survey: "How satisfied were you?" on a 1 to 5 scale. CSAT is the share of respondents choosing 4 or 5.

$$\text{CSAT} = \frac{\text{responses rated 4 or 5}}{\text{total survey responses}} \times 100$$

EXAMPLE 340 of 400 respondents give 4 or 5, so CSAT is 85%. Beware response bias: angry and delighted users answer surveys most.

Task Completion Rate (agents)

The headline agent metric: did the agent reach the goal? Check the final state of the world, not the agent's claim that it succeeded.

$$\text{Task Success} = \frac{\text{tasks whose end state passes the check}}{\text{total tasks}}$$

EXAMPLE On 50 "book a meeting" tasks, the calendar contains a correct event in 41 cases, so success is 82%. Because agents are non-deterministic, run each task several times. pass^k (the chance it succeeds on all k tries) measures reliability, which is what production users feel.

Tool-Call Accuracy

Did the agent pick the right tool, with correct arguments, at the right time? Compare each call with an expected call, or have a judge rate it.

$$\text{Tool Accuracy} = \frac{\text{correct tool calls (right tool and valid arguments)}}{\text{total tool calls}}$$

EXAMPLE 90 calls, 81 correct (right tool, valid JSON, correct parameters) gives 90%. Split it into tool-selection accuracy and argument accuracy to see which part breaks.

Trajectory Efficiency and Step Count

How direct was the path? Step count is simply the number of actions (LLM calls plus tool calls) to finish. Efficiency compares it with a reference or optimal path.

$$\text{Efficiency} = \frac{\text{steps in the optimal trajectory}}{\text{steps the agent took}}$$

EXAMPLE The task needs 4 steps and the agent took 10, looping on the same search twice. Efficiency is 0.40. Judges can also grade the trajectory for unnecessary or risky actions.

Cost per Task

Total spend to complete one task: input and output tokens across every LLM call, plus paid tool and API calls.

$$\text{Cost per Task} = \frac{\sum(\text{token cost} + \text{tool cost})}{\text{tasks attempted}}$$

EXAMPLE 200 tasks cost ₹4,000 in tokens and ₹1,000 in API calls, so ₹25 per task. Also track cost per successful task (if 164 of the 200 succeed, ₹5,000 / 164 = ₹30.5), which punishes expensive failures.

Error Recovery Rate

When something goes wrong (a tool returns an error, a page fails to load, an argument is rejected), how often does the agent recover and still finish?

$$\text{Recovery Rate} = \frac{\text{tasks completed after at least one error}}{\text{tasks that hit at least one error}}$$

EXAMPLE 30 tasks hit an API timeout and 21 still succeed, so recovery is 70%. Test it deliberately by injecting failures.

08 Code Generation and Text-to-SQL

IN THIS SECTION Pass@k | Unit Test Pass Rate | Compile | Edit Similarity | Execution Accuracy | Exact Match | Query Validity | Valid Efficiency Score

> **Code has a superpower other outputs lack: you can run it. The strongest metrics here execute the output and check the result, rather than comparing text.**

Pass@k

The probability that at least one of k generated solutions passes all unit tests. Generate n samples per problem (n ≥ k), count the c that pass, and use the unbiased estimator:

$$\text{pass@}k = \mathbb{E}_{\text{problems}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]$$

EXAMPLE 10 samples, 3 pass. pass@1 = 0.30. pass@5 = 1 − C(7,5) / C(10,5) = 1 − 21/252 = 0.92. pass@1 reflects what a user gets on one try; pass@k with larger k reflects the model's potential when you can generate and test several attempts.

Unit Test Pass Rate

The share of individual test cases passed, which gives partial credit that pass@k does not.

$$\text{Test Pass Rate} = \frac{\text{tests passed}}{\text{total tests}}$$

EXAMPLE A function passes 8 of 10 tests, scoring 0.80, though it still counts as a fail for pass@1. Quality depends on the tests: weak tests reward wrong code, so include edge cases.

Compile (or Parse) Success Rate

The share of generations that compile, or for interpreted languages parse and import without syntax errors. A floor metric: code that does not compile scores zero on everything else.

EXAMPLE 470 of 500 generated Java files compile, so 94%. Useful for catching regressions such as a new prompt that starts wrapping code in markdown fences.

Edit Similarity

For code completion, how close is the suggestion to what the developer finally wrote? Based on Levenshtein edit distance (the minimum number of character insertions, deletions and substitutions to turn one into the other).

$$\text{EditSim} = 1 - \frac{\text{Levenshtein}(\text{generated}, \text{reference})}{\max(|\text{generated}|, |\text{reference}|)}$$

EXAMPLE A 40-character suggestion that needs 4 character edits scores 1 − 4/40 = 0.90. In production, the related acceptance rate (share of suggestions kept) is the business metric.

Execution Accuracy (Text-to-SQL)

Run the generated SQL and the reference SQL on the same database; count it correct if the result sets match.

$$\text{EX} = \frac{\#\{\, i : \text{result}(\hat{Q}_i) = \text{result}(Q_i) \,\}}{N}$$

EXAMPLE 172 of 200 questions return identical results, so EX = 86%. This is the main metric on benchmarks like Spider and BIRD. Run against several database states where possible, because a wrong query can match by luck on one dataset.

Exact Match (Text-to-SQL)

Does the generated query match the reference after normalising formatting (and, in component-level versions, comparing SELECT, WHERE, GROUP BY clauses as sets)? Strict and cheap, but it marks many correct queries wrong, because SQL can be written many valid ways. Example: `WHERE age > 30 AND city = 'Pune'` versus the same conditions in reverse order are equivalent but fail naive exact match.

Query Validity

The share of generated queries that execute without error: valid syntax, real table and column names, correct types.

$$\text{Validity} = \frac{\text{queries that execute without error}}{\text{total queries}}$$

EXAMPLE 190 of 200 run, so 95%. The 10 failures usually point to schema hallucination, such as inventing a `customer_name` column.

Valid Efficiency Score (VES)

From the BIRD benchmark: a correct query also earns credit for being fast. Each correct query is weighted by how its runtime compares with the reference query; incorrect ones score 0.

$$\text{VES} = \frac{1}{N} \sum_{i=1}^{N} 1(\hat{Q}_i \text{ correct}) \cdot \sqrt{\frac{T(Q_i)}{T(\hat{Q}_i)}}$$

EXAMPLE A correct query that takes 4 s when the reference takes 1 s earns √(1/4) = 0.5 instead of 1. On a production warehouse, this is the difference between a ₹10 query and a ₹1,000 one.

09 Reinforcement Learning

IN THIS SECTION Cumulative Reward | Average Episode Return | Sample Efficiency | Convergence Stability | Success Rate

> **An RL agent learns by trial and error to maximise reward. Evaluate how much reward it earns, how fast it learns, and how reliably it does so across random seeds. Always report results over several seeds (5 or more) with confidence intervals, because RL is notoriously noisy.**

Cumulative Reward (Return)

The total reward collected over one episode, often with future rewards discounted by a factor γ between 0 and 1.

$$G = \sum_{t=0}^{T} \gamma^t \, r_t$$

EXAMPLE A game agent earns rewards 1, 0, 2, 5 over four steps. Undiscounted return is 8. With γ = 0.9 it is 1 + 0 + 1.62 + 3.65 = 6.27. Discounting values getting reward sooner.

Average Episode Return

The mean undiscounted return over many evaluation episodes, run with exploration switched off. This is the headline number in most RL papers.

$$\bar{G} = \frac{1}{N} \sum_{i=1}^{N} G_i$$

EXAMPLE 100 evaluation episodes of a balancing task average 487 out of a maximum 500. Report the standard deviation too: 487 ± 5 and 487 ± 120 are very different agents.

Sample Efficiency

How much experience the agent needs to reach a target level of performance. Measured as environment steps (or episodes) until the average return first crosses a threshold, or as the area under the learning curve.

EXAMPLE Agent A reaches a return of 450 after 200,000 steps; agent B needs 2 million. A is 10 times more sample-efficient. This matters enormously when experience is expensive, such as real robots or live A/B traffic.

Convergence Stability

Does performance climb and stay up, or collapse and oscillate? Measure the variance of the return over the last part of training, and the spread across seeds.

$$\text{Stability} = \text{Std}\left(\bar{G}_{\text{last } W \text{ checkpoints}}\right)$$

EXAMPLE Across 5 seeds, one run's return crashes from 480 to 50 midway and never recovers. Even if the average looks fine, that is an unstable algorithm. Interquartile mean (IQM) across seeds is a robust summary that limits the effect of outlier runs.

Success Rate

For goal-based tasks, the share of episodes where the agent achieves the goal, regardless of reward shape.

$$\text{Success Rate} = \frac{\text{episodes reaching the goal}}{\text{total episodes}}$$

EXAMPLE A robot arm picks up the cup in 78 of 100 trials, so 78%. Easier to communicate than return, and immune to reward hacking where the agent scores points without doing the actual task.

10 Computer Vision: Object Detection and Image Generation

IN THIS SECTION | IoU | Precision and Recall | mAP | FPS | FID | Inception Score | CLIP Score | Human Preference

> **Detection is judged on finding the right objects in the right place, fast. Generation is judged on realism, diversity and whether the image matches the prompt.**

IoU (Intersection over Union)

How well a predicted bounding box overlaps the true box. The building block of every detection metric.

$$\text{IoU} = \frac{\text{Area}(B_{\text{pred}} \cap B_{\text{true}})}{\text{Area}(B_{\text{pred}} \cup B_{\text{true}})}$$

EXAMPLE Two 10 × 10 boxes offset by 5 pixels horizontally overlap in a 5 × 10 region. IoU = 50 / (100 + 100 − 50) = 0.33. A prediction usually counts as a true positive when IoU ≥ 0.5.

Precision and Recall (detection)

Same formulas as classification, with matching done by IoU. A predicted box with the right class and IoU above the threshold is a TP; any other prediction is an FP; any true object left unmatched is an FN. Example: 10 people in an image, 9 boxes predicted, 8 match, so precision is 8/9 = 0.89 and recall is 8/10 = 0.80.

mAP (mean Average Precision)

For each class, rank predictions by confidence, build the precision-recall curve, and take the area under it (AP). Average AP across all classes to get mAP.

$$\text{mAP} = \frac{1}{C} \sum_{c=1}^{C} \text{AP}_c$$

! Naming matters: mAP@0.5 uses one IoU threshold of 0.5 (the older Pascal VOC style). COCO's mAP@[.5:.95] averages over ten thresholds from 0.50 to 0.95, so it also rewards tight boxes and runs much lower. Example: AP of 0.72 for cars, 0.58 for people and 0.40 for bicycles gives mAP = 0.57.

FPS (Frames per Second)

How many images the model processes per second, end to end (preprocessing, inference and post-processing such as non-max suppression). Always state the hardware and batch size.

$$\text{FPS} = \frac{\text{frames processed}}{\text{elapsed seconds}}$$

EXAMPLE 1,800 frames in 60 s on one GPU is 30 FPS, which is real-time for most video. Latency per frame (here 33 ms) matters more than FPS for robotics and driving.

FID (Fréchet Inception Distance)

Compares the distribution of generated images with real images. Pass both sets through an Inception network, fit a Gaussian (mean and covariance) to each set of features, and measure the distance between the two Gaussians. Lower is better.

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}\left(\Sigma_r + \Sigma_g - 2\left(\Sigma_r \Sigma_g\right)^{1/2}\right)$$

It captures both quality and diversity: blurry images shift the mean, and a model that produces the same face every time shrinks the covariance. Use at least 10,000 images; FID is biased upward with small samples, and only scores computed the same way are comparable.

Inception Score (IS)

An older metric: an Inception classifier should be confident about each image (it clearly shows something) while predictions across all images should spread over many classes (variety). Higher is better.

$$\text{IS} = \exp\left(\mathbb{E}_x\left[D_{KL}\left(p(y \mid x) \,\|\, p(y)\right)\right]\right)$$

! Limitations: it never looks at real images, and it only understands ImageNet's 1,000 classes, so it says little about faces, art or text-to-image prompts. Prefer FID, and report IS mainly for comparison with older papers.

CLIP Score

How well a generated image matches its text prompt. Embed both with CLIP and take the cosine similarity, usually scaled.

$$\text{CLIPScore} = w \cdot \max\left(\cos(E_{\text{image}}, E_{\text{text}}), 0\right)$$

w is a scaling constant (2.5 in the original paper; some tools scale by 100). Example: "a red bicycle leaning on a blue wall" scores high for a matching image and lower for a blue bicycle. It is weak on counting, spatial relations and text in images, so pair it with human checks.

Human Preference

Show people two images (or outputs) for the same prompt and ask which is better. Report the win rate, or Elo-style ratings across many models, as public image arenas do.

$$\text{Win Rate} = \frac{\text{wins} + 0.5 \times \text{ties}}{\text{total comparisons}}$$

EXAMPLE Model A beats model B in 620 of 1,000 comparisons with 80 ties, giving (620 + 40) / 1,000 = 66%. Learned preference models (trained on such votes) can approximate this cheaply.

11 Clustering

IN THIS SECTION | Silhouette Score | Davies-Bouldin Index | Calinski-Harabasz Index | Adjusted Rand Index

> **Clustering usually has no ground-truth labels, so most metrics judge the shape of the clusters themselves: tight inside, well separated outside. When labels do exist, compare against them with ARI.**

Silhouette Score

For each point, compare how close it is to its own cluster (a) with how close it is to the nearest other cluster (b). Average across all points. Ranges from −1 to 1; higher is better.

$$s(i) = \frac{b(i) - a(i)}{\max\left(a(i), b(i)\right)}$$

Here a(i) is the mean distance from point i to others in its cluster, and b(i) the mean distance to points in the nearest other cluster. Example: a = 2 and b = 8 gives s = (8 − 2) / 8 = 0.75, a well-placed point. Near 0 means the point sits on a boundary; negative means it is probably in the wrong cluster. A common use: run k-means for k = 2 to 10 and pick the k with the highest average silhouette.

Davies-Bouldin Index

For each cluster, find the most similar other cluster, where similarity is the combined spread of the two divided by the distance between their centres. Average across clusters. Lower is better, with 0 as the ideal.

$$\mathrm{DB} = \frac{1}{k}\sum_{i=1}^{k} \max_{j \neq i} \frac{\sigma_i + \sigma_j}{d(c_i, c_j)}$$

σ_i is the average distance of cluster i's points to its centre c_i. Example: two clusters with spreads 1 and 2 whose centres are 10 apart give (1 + 2) / 10 = 0.3, which is good. Move the centres to 3 apart and it becomes 1.0, signalling overlap.

Calinski-Harabasz Index (Variance Ratio Criterion)

The ratio of between-cluster spread to within-cluster spread, adjusted for the number of clusters k and points n. Higher is better.

$$\mathrm{CH} = \frac{\mathrm{tr}(B_k)/(k-1)}{\mathrm{tr}(W_k)/(n-k)}$$

tr(B_k) is the between-cluster dispersion (how far cluster centres sit from the overall centre, weighted by cluster size); tr(W_k) is the within-cluster dispersion (sum of squared distances to each point's own centre). Fast to compute, so it suits large datasets. It favours round, convex clusters, so treat it with caution for odd shapes found by DBSCAN.

Adjusted Rand Index (ARI)

When true labels exist, ARI measures how well the clustering agrees with them by checking every pair of points: are they grouped together in both, or apart in both? It is adjusted for chance, so random clustering scores about 0 and perfect agreement scores 1.

$$\mathrm{ARI} = \frac{\mathrm{RI} - \mathbb{E}[\mathrm{RI}]}{\max(\mathrm{RI}) - \mathbb{E}[\mathrm{RI}]}$$

RI (Rand Index) is the share of point pairs on which the two groupings agree. Example: clustering 1,000 support tickets, then comparing with labels from the support team, gives ARI = 0.68, a solid match. Cluster IDs do not need to match label names; ARI only cares about who is grouped with whom.

12 Production AI

IN THIS SECTION | Latency | Throughput | Cost per Request | Hallucination Rate | Guardrail Violation Rate

> **A model that scores brilliantly offline can still fail users if it is slow, expensive or unsafe. Track these for every system above once it ships.**

Latency (p50, p95, p99)

How long a request takes, reported as percentiles rather than an average. p95 is the time under which 95% of requests finish; the slowest 5% take longer. Sort all request times and read off the value at that position.

$$p95 = \text{the value at position } \lceil 0.95 \times n \rceil \text{ in sorted latencies}$$

EXAMPLE Across 1,000 requests the mean is 800 ms, but p95 is 2.4 s and p99 is 6 s. The mean hides that 1 in 20 users waits three times longer. For LLM apps also track time to first token (how soon text starts streaming), since that is what users perceive as speed.

Throughput

How much work the system handles per unit time: requests per second (RPS), or for LLMs, tokens generated per second.

$$\text{Throughput} = \frac{\text{requests completed}}{\text{time window}}$$

EXAMPLE 18,000 requests in 10 minutes is 30 RPS. Throughput and latency trade off: batching more requests per GPU raises throughput but makes each request wait longer. Load test to find where p95 latency breaks your target.

Cost per Request

Total serving cost divided by requests served. For API-based LLMs, token cost dominates.

$$\text{Cost per Request} = \frac{\text{input tokens} \times p_{\text{in}} + \text{output tokens} \times p_{\text{out}} + \text{infra cost}}{\text{requests}}$$

EXAMPLE An average request uses 3,000 input and 500 output tokens. At illustrative prices of $3 per million input and $15 per million output tokens, that is $0.009 + $0.0075 = $0.0165 per request, or about $16,500 per million requests. Prompt caching, shorter context and routing easy queries to smaller models are the usual levers.

Hallucination Rate (in production)

The same definition as for chatbots, measured continuously on a sample of live traffic by an LLM judge or human spot checks. Watch the trend: a rise after a model, prompt or data change is an early warning.

Guardrail Violation Rate

The share of inputs or outputs that trip a safety or policy check: toxic content, leaked personal data, prompt-injection attempts, off-topic answers, or forbidden advice.

$$\text{Violation Rate} = \frac{\text{responses flagged by guardrails}}{\text{total responses}}$$

EXAMPLE 45 of 100,000 responses are flagged for exposing a phone number, a rate of 0.045%. Track two more numbers with it: how often guardrails block safe content (false refusals, which annoy users) and how much latency the checks add.

One rule ties this handbook together: choose metrics that match what your users care about, measure every stage of the pipeline separately, and check automated metrics against human judgement before you trust them.